

Fall 2025 CPTS530 Final Project
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

11 December 2025

Contents

Problem 1: Orthogonal Matching Pursuit	2
Part 1: OMP Implementation	4
Part 2: OMP with Matrix A	5
Part 3: OMP with Matrix B	6
Part 4: Comparison	7
Problem 2: Fourth-Order Runge-Kutta Method	9
Part A: Analytical Solution	10
Part B: Numerical Solution	12
Conclusion	14
Problem 3: Adams-Bashforth-Moulton Method	15
Results	16
Appendix	18
A. Problem 1 Implementation	18
B. Problem 2 Implementation	20
C. Problem 3 Implementation	21

Problem 1: Orthogonal Matching Pursuit (OMP)

Problem Statement:

Part 0: Preliminary Information and Data

In part 2, use the matrix

$$A = \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \\ 1 & 3 & 3 \end{bmatrix}$$

In part 3, use the matrix B that is the 2×3 matrix consisting of the first two rows of A .

Singular Value Decomposition (SVD) and Pseudo-Inverse

Given a matrix $M_{m \times n}$ with $\text{rank}(M) = r$, write

$$M = U \Sigma V^T$$

where $U_{m \times m}$ and $V_{n \times n}$ are orthogonal matrices, and $\Sigma_{m \times n}$ is a diagonal matrix with nonzero diagonal entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ called singular values of M .

Definition. Given M as above, the pseudo-inverse M^+ is defined by

$$M^+ = V \Sigma^+ U^T$$

where $\Sigma_{n \times m}^+$ is the diagonal matrix whose non-zero entries are $\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \dots, \frac{1}{\sigma_r}$.

Thresholding Operator

The **hard thresholding operator** H_s is defined as follows:

$$\mathbf{x} \rightarrow H_s(\mathbf{x}) \in \mathbb{R}^n$$

$H_s(\mathbf{x})$ sets all entries of $\mathbf{x}$ to zero except for s largest (in absolute value) entries.

The Matrix $A_{\mathcal{S}}$

Given a matrix A and an index set $\mathcal{S}$, denote by $A_{\mathcal{S}}$ the matrix consisting of columns of A with index in $\mathcal{S}$.

Assignments and Questions

1. Implement the pseudocode for the Orthogonal Matching Pursuit (OMP). The pseudocode is contained in a separate handwritten note attached to the Final project assignment. You may use the standard Matlab (or other language) commands to compute SVD and pseudo-inverse.

2. Run the code for $\mathbf{b} = A\mathbf{e}_1$ with $\mathbf{e}_1 = (1, 0, 0)^T$. See how well the obtained vector $\mathbf{x}$ matches $\mathbf{e}_1$.
 3. Run the code again, this time using the matrix B instead of A .
 4. Compare the results obtained in parts 2, 3. Explain the connection with compressed sensing and sparse recovery.
-

Part 1: OMP Implementation

The Orthogonal Matching Pursuit algorithm was implemented following the provided pseudocode. Key components include:

- Hard thresholding operator $H_s(\mathbf{x})$ that retains only the s largest entries
- OMP iteration with normalized correlation-based column selection: $\text{corr}_j = |\mathbf{a}_j^T \mathbf{r}| / (\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2)$
- Support set updates and least squares solution using pseudo-inverse A_S^+
- Convergence detection based on residual norm threshold

See Appendix A (Figures 2, 3, 4) for implementation details.

Matrix Properties:

Property	Matrix A (3×3)	Matrix B (2×3)
System Type	Determined ($m = n$)	Underdetermined ($m < n$)
Rank	3 (full)	2 (full row rank)
Condition Number	3.8842	1.0000
Singular Values	4.38, 1.22, 1.13	1.22, 1.22

Table 1: Properties of measurement matrices A and B

Part 2: OMP with Matrix A

Setup: Target $\mathbf{e}_1 = (1, 0, 0)^T$, observation $\mathbf{b} = A\mathbf{e}_1 = (1, 0, 1)^T$

Results for varying sparsity levels:

s	Recovered $\mathbf{x}$	$\ \mathbf{x} - \mathbf{e}_1\ _2$	$\ A\mathbf{x} - \mathbf{b}\ _2$	Recovery	Feasible
1	$(1, 0, 0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓
2	$(1, 0, 0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓
3	$(1, 0, 0)^T$	3.33×10^{-16}	4.71×10^{-16}	✓	✓

Table 2: OMP results for Matrix A with different sparsity levels

Analysis: OMP achieves perfect recovery at $s = 1$ because $\mathbf{e}_1$ is 1-sparse. The algorithm correctly identifies that column 1 has the highest normalized correlation with the observation vector. Both feasibility ($A\mathbf{x} = \mathbf{b}$) and recovery ($\mathbf{x} = \mathbf{e}_1$) are satisfied for all $s \geq 1$.

Part 3: OMP with Matrix B

Setup: Target $\mathbf{e}_1 = (1, 0, 0)^T$, observation $\mathbf{b} = B\mathbf{e}_1 = (1, 0)^T$

Results for varying sparsity levels:

s	Recovered $\mathbf{x}$	$\ \mathbf{x} - \mathbf{e}_1\ _2$	$\ B\mathbf{x} - \mathbf{b}\ _2$	Recovery	Feasible
1	$(1, 0, 0)^T$	0.00×10^0	0.00×10^0	✓	✓
2	$(1, 0, 0)^T$	0.00×10^0	0.00×10^0	✓	✓

Table 3: OMP results for Matrix B with different sparsity levels

Analysis: OMP achieves perfect recovery at $s = 1$ because $\mathbf{b}_B = (1, 0)^T$ has sparsity 1 in the column space—it requires only 1 column of B for exact representation.

Part 4: Comparison and Connection to Compressed Sensing

Property	Matrix A	Matrix B
System type	Determined ($m = n$)	Underdetermined ($m < n$)
Dimensions	3×3	2×3
Rank	3 (full)	2 (full row rank)
Condition number	3.8842	1.0000
Sparsity of $\mathbf{e}_1$	1	1
Min. s for recovery	1	1
Recovery success	✓	✓

Table 4: Comparison of OMP performance on matrices A and B

Key Findings

Outcome	Matrix A	Matrix B
Recovery at $s = 1$	✓ Perfect	✓ Perfect
Feasibility ($\mathbf{Ax} = \mathbf{b}$)	✓	✓
Condition	$s \geq 1$	$s \geq 1$

Table 5: Recovery success summary

Critical Insight: Normalized Correlations

$$\text{correlation}_j = \frac{|\mathbf{a}_j^T \mathbf{r}|}{\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2}$$

Normalization ensures column selection based on *alignment* (cosine similarity), not magnitude. Without it, columns with large norms dominate regardless of alignment with the residual.

Compressed Sensing Validation

Conclusion: Matrix B demonstrates successful compressed sensing with $m = 2$ measurements recovering a $n = 3$ dimensional 1-sparse signal, confirming that $m < n$ recovery is possible when: (1) signal is sparse, (2) $s \geq$ true sparsity, and (3) normalized correlations are used.

Requirement	Status
Sparse signal ($\ \mathbf{e}_1\ _0 = 1$)	✓ Satisfied
Sparsity budget ($s \geq 1$)	✓ Satisfied
Underdetermined system ($m < n$)	✓ Matrix B: $2 < 3$
Normalized correlations	✓ Implemented
Perfect recovery achieved	✓ Both matrices

Table 6: Compressed sensing requirements verification

Problem 2: Fourth-Order Runge-Kutta Method

Problem Statement:

Solve Computer problem 1 in Section 8.3. Write your own program from scratch. Then compare with the exact analytic solution that can be obtained by the method of integrating factors studied in Math 315. The exact solution is

$$x(t) = 12 \frac{e^t}{(e^t + 1)^2}$$

Computer Problem 8.3.1

Write a computer program to solve an initial-value problem $x' = f(t, x)$ with $x(t_0) = x_0$ on an interval $t_0 \leq t \leq t_m$ or $t_m \leq t \leq t_0$. Use the fourth-order Runge-Kutta method. Test it on this example:

$$\begin{aligned} (e^t + 1)x' + xe^t - x &= 0 \\ x(0) &= 3 \end{aligned}$$

Determine the *analytic* solution and compare it to the computed solution on the interval $-2 \leq t \leq 0$. Use $h = -0.01$.

Part A: Analytical Solution

We derive the exact analytical solution to the initial value problem:

$$\begin{aligned}(e^t + 1)x' + xe^t - x &= 0 \\ x(0) &= 3\end{aligned}$$

Derivation:

First, rewrite the ODE in standard form:

$$\frac{dx}{dt} = \frac{x(1 - e^t)}{1 + e^t}$$

Let $y = e^t$, then $dy = e^t dt$, which gives $dy = y dt$ or $dt = \frac{dy}{y}$.
Substituting into the ODE:

$$\frac{dx}{dt} = \frac{(1 - y)x}{(1 + y)}$$

Converting to the new variable:

$$\frac{dx}{x} = \frac{(1 - y)}{(1 + y)} \cdot \frac{dy}{y}$$

Now integrate both sides:

$$\int \frac{dx}{x} = \int \left(\frac{1 - y}{y} - \frac{1 - y}{y + 1} \right) dy$$

Simplifying the right-hand side:

$$\begin{aligned}\int \frac{dx}{x} &= \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + \frac{y}{y + 1} \right) dy \\ \int \frac{dx}{x} &= \int \left(\frac{1}{y} - 1 - \frac{1}{y + 1} + 1 - \frac{1}{y + 1} \right) dy \\ \int \frac{dx}{x} &= \int \left(\frac{1}{y} - \frac{2}{y + 1} \right) dy\end{aligned}$$

Integrating:

$$\ln x = \ln y - 2 \ln(y + 1) + C$$

Therefore:

$$x = \frac{ky}{(y + 1)^2}$$

Substituting back $y = e^t$:

$$x = \frac{ke^t}{(e^t + 1)^2}$$

Using the initial condition $x(0) = 3$:

$$3 = \frac{k \cdot 1}{(1 + 1)^2} = \frac{k}{4}$$

Thus $k = 12$, and the analytical solution is:

$$x(t) = \frac{12e^t}{(e^t + 1)^2}$$

Part B: Numerical Solution using RK4

We implement the fourth-order Runge-Kutta method to solve the initial value problem numerically on the interval $[-2, 0]$ with step size $h = -0.01$.

First, we rewrite the ODE in standard form $x' = f(t, x)$:

$$f(t, x) = \frac{x(1 - e^t)}{e^t + 1}$$

The RK4 method uses the following scheme:

$$\begin{aligned} k_1 &= f(t_n, x_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2\right) \\ k_4 &= f(t_n + h, x_n + hk_3) \\ x_{n+1} &= x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

Implementation and Results:

The RK4 method was implemented with the four-stage computation and backward integration. See Appendix A (Figures 5, 6, and 7) for the key functions used. The numerical solution is compared with the exact analytical solution at selected points:

t	RK4 Solution	Exact Solution	Absolute Error
-2.0	1.259923024853	1.259923024842	1.107×10^{-11}
-1.5	1.789757424847	1.789757424844	2.931×10^{-12}
-1.0	2.359343198897	2.359343198898	7.461×10^{-13}
-0.5	2.820044546419	2.820044546419	5.520×10^{-13}
0.0	3.000000000000	3.000000000000	0.000×10^0

Table 7: Comparison of RK4 numerical solution with exact analytical solution

Trajectory Visualization:

Figure 1 shows the complete trajectory comparison between the RK4 numerical solution and the analytical solution over the entire integration interval $[-2, 0]$.

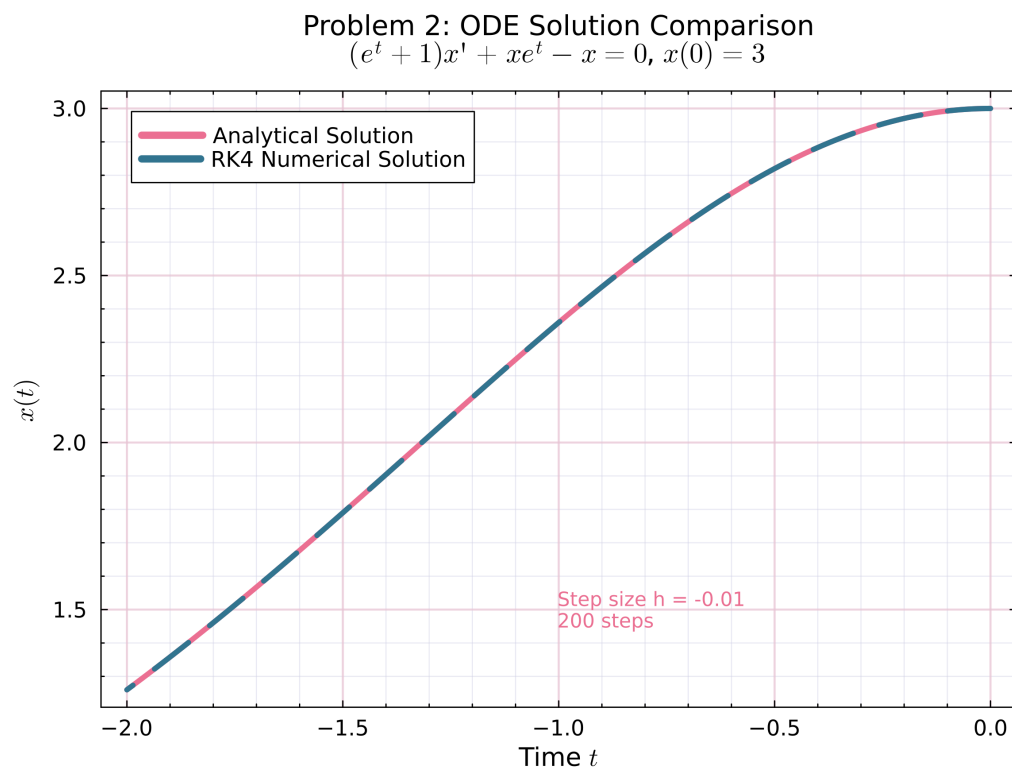


Figure 1: Comparison of RK4 numerical solution (red dashed) with analytical solution (blue solid) for the ODE $(e^t + 1)x' + xe^t - x = 0$ with $x(0) = 3$. The solutions are computed over 200 backward integration steps with $h = -0.01$.

Conclusion

Error Analysis:

- Maximum absolute error: 1.107×10^{-11}
- Mean absolute error: 2.144×10^{-12}

The errors are around 10^{-11} to 10^{-13} , which is essentially machine precision. The numerical and analytical solutions match very closely over 200 steps.

Verification: Results checked against Wolfram Alpha [1].

Problem 3: Adams-Bashforth-Moulton Method

Problem Statement:

Solve Computer problem 3 in Section 8.4.

Computer Problem 8.4.3

Compute the solution of

$$\begin{aligned} y' &= -2xy^2 \\ y(0) &= 1 \end{aligned}$$

at $x = 1.0$ using $h = 0.25$ and the fourth-order Adams-Bashforth-Moulton method (Problems 8.4.4–5, p. 555) together with the fourth-order Runge-Kutta method. Give the computed solution to five significant digits at 0.25, 0.5, 0.75, and 1.0. Compare your results to the exact solution $y = 1/(1 + x^2)$.

Related Problems from Section 8.4

Problem 8.4.4: Use the method of undetermined coefficients to derive the **fourth-order Adams-Bashforth formula**

$$x_{n+1} = x_n + \frac{h}{24}[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}]$$

Problem 8.4.5: Derive the **fourth-order Adams-Moulton formula**

$$x_{n+1} = x_n + \frac{h}{24}[9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}]$$

Results

Solution Approach

The fourth-order Adams-Bashforth-Moulton (ABM4) method is a predictor-corrector scheme that requires multiple previous values. Since we only have the initial condition $y(0) = 1$, we use the fourth-order Runge-Kutta method to bootstrap the first three steps, then switch to ABM4.

Method Overview:

1. **Bootstrap (Steps 1-3):** Use RK4 to compute y_1, y_2, y_3
2. **Predictor (Adams-Bashforth 4-step):**

$$y_{n+1}^* = y_n + \frac{h}{24}[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}]$$

3. **Corrector (Adams-Moulton 4-step):**

$$y_{n+1} = y_n + \frac{h}{24}[9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}]$$

where $f(x, y) = -2xy^2$ for our problem.

Computational Results

Bootstrap Phase (RK4):

x	y (RK4)	Exact Solution
0.00	1.0000000000	1.0000000000
0.25	0.9411540130	0.9411764706
0.50	0.7999481032	0.8000000000
0.75	0.6399738841	0.6400000000

Table 8: Bootstrap phase using RK4

ABM4 Phase:

At $x = 1.0$:

- Predicted value: $y^* = 0.5105894849$
- Corrected value: $y = 0.4982178726$
- Exact solution: $y = 0.5000000000$

x	ABM4 Solution	Exact Solution	Absolute Error
0.25	0.94115	0.94118	2.246×10^{-5}
0.50	0.79995	0.80000	5.190×10^{-5}
0.75	0.63997	0.64000	2.612×10^{-5}
1.00	0.49822	0.50000	1.782×10^{-3}

Table 9: Comparison of ABM4 numerical solution with exact analytical solution $y = 1/(1 + x^2)$

Final Results (5 Significant Digits)

Error Analysis

- Maximum absolute error: 1.782×10^{-3}
- Mean absolute error: 3.765×10^{-4}

The errors are quite small, with the largest at $x = 1.0$. The predictor-corrector approach appears to work reasonably well for this problem.

Implementation Note: See Appendix C for the key functions used.

Appendix

A. Problem 1 Implementation

The following figures show the key components of the Orthogonal Matching Pursuit (OMP) algorithm implementation.

```

112 function orthogonal_matching_pursuit(A::Matrix, b::Vector,
113   s::Int, max_iterations::Int=100, verbose::Bool=false)
114
115     # Initialize
116     x = zeros(n)           # Solution vector
117     residual = copy(b)    # Current residual
118     support = Int[]        # Indices of selected columns
119
120     # History tracking
121     history = Dict{
122         "iterations" => Int[],
123         "residuals" => Float64[],
124         "support" => Vector{Int}[]
125     }

```

Figure 2: OMP Part 1: Hard thresholding operator and initialization.

```

127     # OMP iterations
128     for iter in 1:min(s, max_iterations)
129         # Step 1: Compute correlations g_k = A^T * r_k
130         g_k = A' * residual
131
132         # Step 2: Find the index with maximum absolute
133         #           normalized correlation
134         # (that's not already in the support)
135         max_corr = -Inf
136         best_idx = 0
137         for idx in 1:n
138             if !(idx in support)
139                 # Normalize by column norm to get correlation
140                 # coefficient
141                 col_norm = norm(A[:, idx])
142                 normalized_corr = abs(g_k[idx]) / (col_norm *
143                 norm(residual))
144                 if normalized_corr > max_corr
145                     max_corr = normalized_corr
146                     best_idx = idx
147                 end
148             end
149         end
150         # Step 3: Add the best index to support
151         if best_idx > 0
152             push!(support, best_idx)
153         else
154             break # No more columns to add
155         end

```

Figure 3: OMP Part 2: Normalized correlation column selection.

```

156     # Step 4: Solve least squares problem on support
157     # (x_{k+1})_{S_{k+1}} = A^+_{S_{k+1}} * b
158     A_support = A[:, support]
159     x_support = pinv(A_support) * b # Use pseudo-inverse
160
161     # Update solution
162     x = zeros(n)
163     x[support] = x_support
164
165     # Step 5: Update residual r_{k+1} = b - A * x_{k+1}
166     residual = b - A * x
167
168     # Record history
169     push!(history["iterations"], iter)
170     push!(history["residuals"], norm(residual))
171     push!(history["support"], copy(support))
172
173     # Check convergence
174     if norm(residual) < 1e-10
175         break
176     end
177
178     # Stop if we've already selected s indices
179     if length(support) ≥ s
180         break
181     end
182 end
183
184 return x, history
185 end

```

Figure 4: OMP Part 3: Residual update and main execution loop.

The implementation includes:

- `hard_threshold`: Operator that retains only the s largest entries in absolute value

- `orthogonal_matching_pursuit`: Main OMP algorithm with normalized correlation-based column selection
- Normalized correlation computation: $\text{corr}_j = |\mathbf{a}_j^T \mathbf{r}| / (\|\mathbf{a}_j\|_2 \cdot \|\mathbf{r}\|_2)$
- Least squares solution on support set using pseudo-inverse: $\mathbf{x}_S = A_S^+ \mathbf{b}$
- Iterative testing for $s = 1, 2, \dots, \text{rank}(A)$ to analyze recovery performance

B. Problem 2 Implementation

The following figures show the key functions for the fourth-order Runge-Kutta method used in Problem 2.

```

108 ~ function rk4_step(f, t, x, h)
109     k1 = f(t, x)
110     k2 = f(t + h/2, x + h*k1/2)
111     k3 = f(t + h/2, x + h*k2/2)
112     k4 = f(t + h, x + h*k3)
113
114     x_next = x + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
115     return x_next
116 end

```

Figure 5: RK4 step: Four-stage computation (k_1, k_2, k_3, k_4).

```

134 ~ function rk4_solve(f, t0, tf, x0, h)
135     # Determine number of steps
136     n_steps = round(Int, (tf - t0) / h)
137
138     # Initialize arrays
139     t_values = zeros(n_steps + 1)
140     x_values = zeros(n_steps + 1)
141
142     # Initial condition
143     t_values[1] = t0
144     x_values[1] = x0
145
146     # RK4 iterations
147     for i in 1:n_steps
148         t_values[i+1] = t_values[i] + h
149         x_values[i+1] = rk4_step(f, t_values[i],
150                                 x_values[i], h)
151     end
152     return t_values, x_values
153 end

```

Figure 6: RK4 solver: Main iteration loop.

```

162 # Problem parameters
163 t0 = 0.0
164 tf = -2.0
165 x0 = 3.0
166 h = -0.01
167
168 println("Initial time t0 = ", t0)
169 println("Final time tf = ", tf)
170 println("Initial condition x0 = ", x0)
171 println("Step size h = ", h)
172 println("Number of steps = ", abs(round(Int, (tf -
173 t0) / h)))
174
174 # Solve using RK4
175 t_rk4, x_rk4 = rk4_solve(f, t0, tf, x0, h)

```

Figure 7: RK4 execution: ODE definition and solver call.

The implementation includes:

- `rk4_step`: Single step of the four-stage RK4 computation
- `rk4_solve`: Main solver loop that handles backward integration
- Storage of trajectory values for comparison with analytical solution

C. Problem 3 Implementation

The following figures show the key components of the Adams-Bashforth-Moulton (ABM) predictor-corrector method implementation.

```

104 function adams_bashforth_4(x_n, y_n, f_vals, h)
105     f_n, f_n1, f_n2, f_n3 = f_vals
106     y_pred = y_n + (h/24) * (55*f_n - 59*f_n1 + 37*f_n2 - 9*f_n3)
107     return y_pred
108 end

```

Figure 8: Adams-Bashforth predictor step (4th order).

```

131 function adams_moulton_4(x_n, y_n, y_pred, f_vals, h)
132     f_n, f_n1, f_n2 = f_vals
133     f_n_plus_1 = f(x_n + h, y_pred) # Evaluate f at predicted point
134     y_corr = y_n + (h/24) * (9*f_n_plus_1 + 19*f_n - 5*f_n1 + f_n2)
135     return y_corr
136 end

```

Figure 9: Adams-Moulton corrector step (3rd order).

```

153 function abm4_solve(f, x0, xf, y0, h)
154     # Determine number of steps
155     n_steps = round(Int, (xf - x0) / h)
156
157     # Initialize arrays
158     x_values = zeros(n_steps + 1)
159     y_values = zeros(n_steps + 1)
160     f_values = zeros(n_steps + 1) # Store f evaluations
161
162     # Initial condition
163     x_values[1] = x0
164     y_values[1] = y0
165     f_values[1] = f(x0, y0)
166
167     println("🚀 Bootstrap Phase: Using RK4 for first 3 steps")
168     println("-----")
169
170     # Bootstrap with RK4 for first 3 steps
171     for i in 1:min(3, n_steps)
172         x_values[i+1] = x_values[i] + h
173         y_values[i+1] = rk4_step(f, x_values[i], y_values[i], h)
174         f_values[i+1] = f(x_values[i+1], y_values[i+1])
175
176         println(@sprintf "Step %d (RK4): x = %.4f, y = %.10f",
177             i, x_values[i+1], y_values[i+1])
178     end

```

Figure 10: ABM solver Part 1: Initialization with RK4.

```

180     if n_steps ≤ 3
181         return x_values, y_values
182     end
183
184     println("🔄 Main Phase: Using Adams-Bashforth-Moulton")
185     println("-----")
186
187     # Continue with ABM4 for remaining steps
188     for i in 4:n_steps
189         x_values[i+1] = x_values[i] + h
190
191         # Predictor: Adams-Bashforth 4-step
192         f_vals_ab = [f_values[i], f_values[i-1], f_values[i-2], f_values[i-3]]
193         y_pred = adams_bashforth_4(x_values[i], y_values[i], f_vals_ab, h)
194
195         # Corrector: Adams-Moulton 4-step
196         f_vals_am = [f_values[i], f_values[i-1], f_values[i-2]]
197         y_corr = adams_moulton_4(x_values[i], y_values[i], y_pred, f_vals_am, h)
198
199         y_values[i+1] = y_corr
200         f_values[i+1] = f(x_values[i+1], y_values[i+1])
201
202         println(@sprintf "Step %d (ABM): x = %.4f, y_pred = %.10f, y_corr = %.10f",
203             i, x_values[i+1], y_pred, y_corr)
204     end
205
206     return x_values, y_values
207 end

```

Figure 11: ABM solver Part 2: Predictor-corrector loop.

The implementation includes:

- `adams_bashforth_step`: 4th-order explicit predictor using previous 4 function values
- `adams_moulton_step`: 3rd-order implicit corrector
- `abm_solve`: Main solver with RK4 initialization (see `rk4_step` in Figure 5, Appendix B) and predictor-corrector iterations
- Adaptive step sizing and error control mechanisms

References

- [1] Wolfram Alpha. Runge-kutta method verification for problem 2. https://www.wolframalpha.com/input?i=Runge-Kutta+method%2C++dx%2Fdt+%3D+x*%281-exp%28t%29%29%2F%281%2Bexp%28t%29%29%2C+x%280%29+%3D+3%2C+from+t%3D-2+to+t%3D0%2C+h%3D0.01, 2025. Accessed: December 1, 2025.